UNIVERSITY PARTNER

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Part 1 : Question and Answer

Student Id : 2408003

Student Name : Sandip Dhakal

Group : L5CG15

Tutor : Jinu Nyachhyon

Submitted on : 10-May-2026

Table of content

Long Question 1: Unsupervised Learning at eSewa

As a data scientist at eSewa, there are two high-value business problems that unsupervised learning can solve: customer segmentation and fraud detection. These problems fit the unsupervised paradigm well since labelled data is scarce and transactions scale into millions daily.

Problem 1: Customer Segmentation

eSewa provides utility-payments services for millions of users whose behaviours vary greatly. Some use our channel to pay electricity bills every month, while others use it heavily for peer-to-peer payments. There are also inactive customers who make one transaction at the time of onboarding and then never return.

Marketing campaigns currently lack segmentation and push the same offer to all users, wasting budget and resulting in poor engagement.

I would apply clustering algorithms here. K-Means is my go-to baselinewriter because it scales to millions of transactions and works great on large table data. When the clusters are non-spherical or there are noisy points to be ignored, DBSCAN is my next choice. Features like frequency, average ticket size, merchant category distribution, time-of-day behaviour, and recency can lead to meaningful clusters. PCA may also be applied beforehand to remove correlated features and reduce dimensionality.

Every day, the batch pipeline can write cluster ids to a user-segment dimension table in the DW. These segments can then be used by marketing partners for targeted offers, or the product team can customize dashboards and push notifications based on segments. Segmenting customers is a classic batch use case since it does not have to be updated every second.

Limitation: Choosing the optimal k for k-means is non-trivial, and silhouette scores computed on transaction data tend to be noisy. Segments should always be sanity checked with business teams before deploying customer-facing campaigns.

Problem 2: Fraud Detection

Fraud is an inherently unsupervised problem because labels are missing or very delayed, and true fraud cases are also extremely rare. Because of this, models cannot learn fraud patterns successfully in a supervised manner. However, if we fit the model on “normal” transactions, then anomalies should stand out.

Isolation Forest is my first choice here followed by something like an Autoencoder that is trained exclusively on verified legitimate transactions. The advantage of an Autoencoder is that the output layer produces an anomaly score based on reconstruction error rather than just a binary label. The business can benefit from this anomaly score to create risk-based thresholds.

In production, I would use something like this as a real-time scoring service. All transactions can be scored prior to approval, and transactions exceeding a certain threshold can be sent for OTP verification or manual review depending on the score. Model training can happen weekly in batch on the latest set of verified legitimate transactions.

Limitation: Autoencoders experience catastrophic drift if user behaviour changes slowly over time. For example, during festive seasons like Dashain or Tihar when transaction volumes are high and usual behaviour changes. Monitor the reconstruction-error distribution carefully and force retrain if it shifts too much.

Short Question 1

(Dropout and early stopping): Give two techniques that can reduce overfitting in deep learning models. For each technique, describe how it works and give a real-world example.

Dropout

In dropout, we randomly set the activations of neurons to zero during training. Effectively, this creates an ensemble of deep nets at training time which are averaged at test time. Dropout forces the network to learn redundant representations since every iteration it drops a random subset of neurons. Since individual neurons cannot depend on activation of others, the model becomes more robust and less likely to overfit. Slows training by a small amount since each iteration now trains only a sub-network.

Example: Classification of images using CNNs such as cat vs dog classifier.

Early Stopping

Rather than train for some arbitrary number of epochs, we validate after every epoch and stop training when validation error begins to rise. Using just training data, the model will keep reducing error by starting to memorise the training data points. Since validation data is not seen by the model during training, the first epoch where validation error increases is good enough to prevent overfitting. Drawback is that we need a validation set. Training data is reduced by the size of the validation set.

Example: Fraud detection on imbalanced datasets where early stopping will prevent the network from memorising fraud examples.

Short Question 2

(CNN vs RNN): Compare and contrast CNNs and RNNs in architecture and application.

Convolutional Neural Networks are mostly applied to visual data such as images or videos, while Recurrent Neural Networks deal with sequential data such as text, speech or time-series. Here are some high-level difference between the two:

Input Structure and Type of Data

CNNs are applied to inputs that have some sort of grid-like structure and where neighbouring elements are correlated. ie. height x width in images and time x frequency in spectrograms. RNNs are applied to sequential inputs where the position of the inputs matters and the network needs to carry information across the sequence.

Information Flow

CNNs have strictly forward flow of information by applying filters across height and width dimensions. Additionally, pooling layers further condense the inputs by simply applying a summary statistic. Whereas in RNNs, the hidden state at time t is a function of hidden state at time t – 1.

Weight Sharing

CNNs have weight sharing across space. The same convolutional filter applied across height and width dimensions of the image. RNNs have weight sharing across time. Same weight matrix applied at every timestep of the sequence.

Examples

CNN examples include identifying crop disease from leaf images, classifying skin-cancer using dermatological photos etc. RNN examples include Nepali speech-to-text, English-to-Nepali translation engine, stock-price prediction etc.

Typical Training Problems and Solutions

CNNs: Exploding/vanishing gradients if the network is very deep. Gradient clipping, batch normalisation, skip connections can help.

RNNs: Exploding/vanishing gradients if the network is deep in time. Use LSTM/GRU cells with gating mechanisms. Additionally, all neural nets suffer from overfitting if model capacity is too high. Use dropout, data augmentation, L2 regularisation, early stopping.